ActiMM: Worklist-Driven Sparse Convolution Runtime on GPUs

Anonymous Author(s)

Abstract

Sparse convolution is the dominant primitive for 3D point-cloud perception, yet existing GPU runtimes face a fundamental tradeoff between *computation skipping* and *execution efficiency*. Gather-GEMM-scatter approaches skip empty neighbors but suffer from per-offset launch overhead and load imbalance. Implicit GEMM approaches fuse all offsets into one efficient kernel but cannot skip empty neighbors, wasting compute proportional to sparsity.

We present ActiMM, a sparse convolution runtime built on *worklist-driven execution*. ActiMM precomputes all active neighbor pairs, groups them by kernel offset, and flattens the result into a contiguous tile stream executed by a single persistent-thread kernel. Empty offsets are skipped entirely; the stride loop provides automatic global load balance; and each tile is a pure dense GEMM. To recover data locality without heavy scheduling, ActiMM applies offset-major contiguous tile assignment and autotuned weight-stationary micro-kernels.

We evaluate ActiMM end-to-end on 3D object detection and semantic segmentation models across real-world point-cloud datasets. ActiMM achieves **TODO**× end-to-end speedup over TorchSparse++ on **TODO** and **TODO**× over SpConv v2 on **TODO**, while producing bit-identical results under FP32.

1 Introduction

Sparse convolution has become a key computational primitive in 3D point cloud perception, playing a central role in deep learning applications such as autonomous driving, robotics, and AR/VR. The importance of sparse convolution stems from the intrinsic data spatial sparsity of these applications: input data features are represented on high-dimensional voxel grids, while only a tiny fraction of voxels are active. For example, in LiDAR-based detection benchmarks such as KITTI, the active ratio is often around 0.01%, and even in indoor scenes or CAD models, it is typically only about 1%. This sparsity implies that directly applying dense convolution would waste substantial computation and memory on empty regions. Therefore, the central challenge in sparse convolution is not the convolution operation itself, but how to efficiently identify, organize, and execute the small number of irregularly distributed, input-dependent computations on GPUs. Efficient sparse convolution is thus essential for satisfying the stringent latency requirements of real-world deployment.

Unlike sparse matrix multiplication, sparse convolution is challenged not only by sparsity itself but by the fact that its computation is organized around runtime-defined spatial neighborhoods. For example, a 3×3×3 convolution has its each output location associated with 27 possible spatial offsets, but under low-density inputs, most offsets at most locations correspond to empty neighbors. Computation therefore occurs only on realized input–output adjacencies, or pairs, where an active output voxel finds an active input voxel at a given offset. Each offset shares a dense weight matrix across all such pairs, so the operator can be viewed as repeatedly applying offset-specific shared weights over a dynamically generated set of spatial pairs. Crucially, these adjacencies are not statically known but are induced by the runtime spatial distribution of active voxels. As a result, sparse convolution must simultaneously handle sparse computation, dynamic neighborhoods, indirect memory accesses, and highly uneven workload distribution, which together make efficient GPU execution fundamentally difficult.

Existing GPU sparse convolution engines can be broadly categorized into two execution strategies, including output-stationary and weight-stationary. Weight-stationary engines such as MinkowskiEngine partition computation by kernel offset, dispatching a dedicated matrix multiplication over only the valid input–output pairs at each offset. This avoids wasted arithmetic on empty voxels and the corresponding weight slice is naturally reused across all pairs within each offset. However, the number of active pairs varies widely across offsets due to intrinsic spatial sparsity, fragmenting the workload into many unevenly sized tasks that cannot saturate the GPU uniformly and leaving computational throughput well below the hardware peak. In contrast, output-stationary engines such as SpConv v2 and TorchSparse++ mirror the implicit-GEMM of dense convolution. Each thread block iterates over all kernel offsets to accumulate partial sums for its assigned output voxels in place, delivering higher throughput with more uniform occupancy. This regularity, however, means every potential neighbor position is processed uniformly, including the many that are empty, so a large fraction of multiply accumulate operations produce and discard zeros.

Overall, although the two strategies suffer from ostensibly different bottlenecks, neither succeeds in converting the GPU's raw execution capability into effective throughput, i.e., the fraction of GPU peak throughput that contributes to valid input-output pairs. Weight-stationary engines perform almost exclusively effective arithmetic, yet their raw throughput is held back by an inability to uniformly distribute work across the GPU, leaving effective throughput low in absolute terms (xx to xx% of the peak throughput

in our evaluation). Output-stationary engines sustain high raw throughput, but waste a significant share of it on empty neighbor positions, so their effective throughput remains limited (xx to xx% of the peak throughput in our evaluation). The core challenge of low-density sparse convolution is therefore not to maximize raw throughput alone, but to elevate effective throughput.

To improve effective GPU throughput, a natural alternative is to discard the structural assumptions of conventional convolution and instead extract all active input–output pairs into a single and uniform execution stream. Because only nonzero pairs are scheduled, this strategy avoids both the workload fragmentation of the weight-stationary strategy and the wasted computation of the output-stationary strategy. The collected pairs are then partitioned into tiles, i.e., equally-sized chunks of work each assigned to one GPU thread block, enabling even distribution across thread blocks for global load balance. However, interleaving tiles with different kernel offsets destroys weight reuse between adjacent tiles and prevents partial sums for the same output voxel from being accumulated in registers, thereby requiring multiple scattered atomic updates to global memory. Fundamentally, this stems from the GPU execution model: each thread block processes a single tile and retires, while the hardware scheduler alone determines where subsequent tiles execute. The result is that, although the total arithmetic is by construction proportional to the number of active pairs, the realized effective throughput is bounded not by compute capacity but by locality losses on the weight, input, and output sides. Counterintuitively, this bottleneck intensifies as the active ratio increases: a higher active ratio implies more simultaneously live kernel offsets and output locations, thereby amplifying the cost of each missed reuse opportunity.

Based on the above insights, we propose ActiMM, a worklist-driven sparse convolution runtime that computes on genuinely active input–output pairs while navigating the trade-off between global load balance and data locality recovery. ActiMM begins by explicitly enumerating every valid pair across all kernel offsets and flattening them into a single, unified worklist. The worklist is then partitioned into fixed-size tiles, each of which is a batch of pairs from one offset and serves as the basic GPU scheduling unit. A persistent-thread execution model in ActiMM lets each thread block consume multiple tiles in stride, enabling locality-aware reordering without disrupting the tiled execution strategy. ActiMM further employs an offline analyzer to select the optimal locality optimization strategy for different input activation ratios. At low active ratios, offset-major ordering arranges tiles by kernel offset so that consecutive tiles processed by the same thread block can reuse weights, restoring weight locality with negligible overhead. At high active ratios, static bundling groups tiles from different offsets that contribute to the same output positions, promoting an output-stationary execution pattern and reducing redundant global memory traffic.

Experimental results show ...

Our contribution

2 Background and Motivation

This section formalizes the sparse convolution operator (§2.1), surveys the two dominant execution strategies and their limitations (§2.2), positions ActiMM relative to orthogonal optimization directions (§2.3), and quantifies the effective throughput gap that motivates our design (§2.4).

2.1 Sparse Convolution Primer

Computation model. Let $\mathcal{A}$ denote the set of *active* voxels in a high-dimensional grid (e.g., a voxelized LiDAR scan). A sparse convolution with kernel volume $K = k^3$ defines K spatial offsets $\{d_1, \ldots, d_K\}$. For each active output voxel $p \in \mathcal{A}_{\text{out}}$ and each offset d, if $p + d$ is also active in the input set $\mathcal{A}_{\text{in}}$, the pair $(p,\ p{+}d,\ d)$ is an *active pair*. The convolution computes

$$\mathbf{y}[p] = \sum_{d:\,(p+d)\in\mathcal{A}_{\text{in}}} \mathbf{W}[d]\ \mathbf{x}[p{+}d], \tag{1}$$

where $\mathbf{W}[d] \in \mathbb{R}^{C_{\text{out}}\times C_{\text{in}}}$ is the dense weight matrix for offset d, and $\mathbf{x}$, $\mathbf{y}$ are the input and output feature vectors of width C_{in} and C_{out}, respectively. The total computation of the operator is determined by the number of active pairs, not by the grid volume.

Submanifold vs. full convolution. Two variants are common in practice. *Submanifold* (SubM) convolution restricts the output active set to equal the input active set ($\mathcal{A}_{\text{out}} = \mathcal{A}_{\text{in}}$), so the pair structure is identical across consecutive SubM layers and can be computed once and reused. *Full* convolution applies stride-2 downsampling; the output active set changes, requiring the pair structure to be rebuilt for each layer. In typical detection backbones (e.g., SECOND [8], VoxelNeXt [1]), SubM layers constitute the vast majority of sparse convolution layers, making pair-structure reuse an important practical consideration.

Distinction from sparse matrix multiplication. Sparse matrix multiplication (SpMM) operates on sparsity in the *channel* dimension: the nonzero pattern is static or fixed within a batch and is compactly represented by CSR/CSC formats. Sparse convolution, by contrast, exhibits sparsity in the *spatial* dimension: the nonzero pattern is determined by the input point cloud and differs for every input. Moreover, the computation is not a single sparse matrix–vector product but K groups of active pairs, each sharing a distinct dense weight matrix $\mathbf{W}[d]$. Standard sparse-matrix techniques (e.g., cuSPARSE SpMM) therefore do not directly apply; sparse convolution requires a dedicated execution strategy.

2.2 Existing Execution Strategies

Weight-stationary. The earliest sparse convolution engines, exemplified by MinkowskiEngine [2], partition computation by kernel offset. For each offset d, the engine gathers the active pairs into a contiguous buffer, invokes a dense GEMM over those pairs, and scatters the results back to the output feature map. This strategy performs *exact work*: offsets with no active pairs are skipped entirely, and the weight matrix $\mathbf{W}[d]$ is naturally reused across all pairs within each GEMM call. However, the number of active pairs varies widely across offsets due to the spatial sparsity of the input, fragmenting the workload into up to K unevenly sized tasks. At low active ratios, many offsets produce only a handful of pairs, yielding thin GEMMs that cannot saturate the GPU. The resulting K separate kernel launches further add dispatch overhead. Subsequent systems (TorchSparse [5] adaptive grouping, SpConv v1 fused dispatch, fused gather/scatter to eliminate intermediate buffers) mitigate launch overhead and data-movement cost, but the root cause—per-offset independent execution—remains. The net effect is exact arithmetic but low raw throughput.

Output-stationary. To address the fragmentation of weight-stationary execution, output-stationary engines such as SpConv v2 [3] and TorchSparse++ [7] mirror the implicit-GEMM strategy of dense convolution. The input features for all K offsets are logically concatenated along a K-dimension, forming an equivalent GEMM in which each thread block iterates over all offsets to accumulate partial sums for its assigned output voxels in place. This eliminates per-offset launch overhead and provides uniform occupancy across thread blocks. Output partial sums are accumulated in registers without atomic operations, preserving output locality. The cost, however, is that the K-dimension necessarily includes empty neighbor positions: at an active ratio of 1%, a typical output voxel in a 3×3×3 kernel has fewer than 2 active neighbors out of 27, meaning over 90% of the multiply–accumulate operations produce and discard zeros. TorchSparse++ introduces a bitmask-based sort to enable warp-level K-tile skipping, but the skip granularity is limited by the alignment of empty patterns within a warp; at low densities the realized skip rate falls well short of the theoretical maximum. The net effect is high raw throughput but substantial wasted computation.

2.3 Orthogonal Optimization Directions

Several lines of work improve sparse convolution performance through mechanisms orthogonal to the execution strategy itself. *Compiler-based* optimizations (TorchSparse++ TVM codegen [7], PCEngine [6], PointAcc [4]) apply operator fusion, data-layout selection, and hardware-specific code generation within an existing execution model, improving implementation quality without changing the fundamental scheduling strategy. *Preprocessing* optimizations (e.g., SpConv v2's GPU hash-table acceleration [3]) reduce the cost of building the neighbor map but do not affect how the convolution kernel itself executes. *Hardware acceleration* approaches (e.g., PointAcc's custom accelerator [4]) change the compute substrate but not the software-level execution model. All of these directions are orthogonal to ActiMM: ActiMM proposes a new execution model, and the above optimizations remain applicable on top of it.

2.4 The Effective Throughput Gap

Metric definition. We define *effective computation throughput* as

$$T_{\text{eff}} = \frac{N_{\text{pairs}} \times C_{\text{in}} \times C_{\text{out}} \times 2}{t_{\text{wall}}}, \tag{2}$$

where N_{pairs} is the number of active pairs, C_{in} and C_{out} are the channel widths, the factor of 2 accounts for multiply and accumulate, and t_{wall} is the wall-clock kernel time. Unlike raw throughput (which includes wasted FLOPs on empty neighbors), effective throughput counts only computation that contributes to the final result. For weight-stationary engines the two metrics are approximately equal (no wasted work) but the absolute value is low due to fragmentation. For output-stationary engines the raw throughput is high but the effective throughput is substantially lower due to wasted computation on empty positions. This metric provides a unified comparison basis across the two strategy families.

Quantifying the gap. Across the realistic active-ratio range (0.01%–30%), both output-stationary (TorchSparse++) and weight-stationary (MinkowskiEngine) engines achieve only a small fraction of the GPU's peak throughput. At 1% active ratio—typical for outdoor LiDAR—the best existing system reaches less than **XX**% of peak effective throughput. The gap persists even at higher densities: at 10%, effective throughput remains below **XX**% of peak. This quantifies the qualitative claim from §1: neither strategy family converts the GPU's raw capability into useful work efficiently.

Opportunity. The two strategy families converge to the same outcome through different paths: one has proportionality (exact work) but lacks throughput; the other has throughput but lacks proportionality. No existing execution model maximizes effective throughput across the density spectrum. ActiMM targets this gap: at low densities, exact work eliminates wasted computation; at high densities, a controlled amount of redundant computation is traded for higher raw throughput; in both regimes, the system selects the configuration that maximizes effective throughput.

3 Overview

This section establishes the high-level picture of ActiMM: the target deployment scenario (§3.1), the execution model (§3.2), and the density-aware configuration mechanism (§3.3). Detailed design is deferred to §**??**.

3.1 Working Scenario

ActiMM targets *inference* on 3D point-cloud models. The input is a single frame (e.g., one LiDAR scan), which is voxelized and fed into a backbone composed of multiple sparse convolution layers (e.g., SECOND [8], VoxelNeXt [1], SparseResUNet42 [2]). Sparse convolution dominates the inference time of these models. Because each input frame produces a different set of active voxels, the computation graph of every sparse convolution layer is *input-dependent*.

The inference setting imposes three key constraints that shape ActiMM's design. First, *latency sensitivity*: autonomous-driving pipelines require real-time response, so per-frame overhead must be minimized. Second, *batch size one*: inference typically processes a single frame at a time, leaving no opportunity to amortize scheduling cost across a batch. Third, *fixed model and hardware*: the same model is deployed on the same GPU to process a stream of different inputs. This last property makes *offline profiling* practical—the model architecture and hardware are known in advance—while the first two properties make *runtime schedule construction* expensive, since its cost cannot be hidden or amortized.

3.2 Execution Model

Core abstraction: the worklist. ActiMM reorganizes the irregular computation of sparse convolution into a *flat tile stream*. For each active output voxel and each of its active input neighbors, the system generates an active pair; all pairs are grouped by offset and then sliced into fixed-size tiles, which are concatenated into a single global *worklist*. Each tile is a batch of pairs from the same offset, corresponding to a standard dense GEMM—GPU-friendly and Tensor Core compatible. Offsets with no active pairs produce no tiles and are skipped entirely, so the total computation is proportional to the number of active pairs by construction. Because all tiles have a uniform size, a single stride-loop assignment achieves global load balance without per-offset resource scheduling.

This is the fundamental departure from existing approaches. Output-stationary engines use the *output voxel* as the scheduling unit and iterate over all offsets per voxel. Weight-stationary engines use the *offset* as the scheduling unit and execute each offset independently. ActiMM uses the *tile* as the scheduling unit, pooling the effective work from all offsets into one unified execution stream.

Persistent threads and locality recovery. The worklist determines *what* to compute; the question is *how* to schedule it. Under ordinary grid dispatch, each thread block processes one tile and exits; the hardware scheduler determines where subsequent tiles execute. Logically adjacent tiles may land on different SMs, and register/cache state cannot be preserved across tiles.

ActiMM addresses this with *persistent threads*: each thread block is pinned to an SM and consumes multiple tiles via a stride loop, so that the logical ordering of the worklist directly translates into the physical execution order. On top of this mechanism, ActiMM provides two density-dependent locality recovery strategies—not two separate systems, but two *operating points* of the same persistent-thread execution model:

- **Low density: offset-major ordering.** Tiles from the same offset are placed consecutively in the worklist. A persistent thread block processes them in sequence, keeping the weight matrix fragment $\mathbf{W}[d]$ live in registers across tiles. This restores weight locality with zero redundant computation.
- **High density: static offset bundles.** Several offsets are grouped into a *bundle*. Within a bundle, tiles that contribute to the same output voxels are co-scheduled so that partial sums can be accumulated in registers before a single write to global memory, partially restoring output locality. The cost is a small amount of redundant computation for inactive offsets within a bundle, traded for higher raw throughput.

At low active ratios, bundle size degenerates to 1 and the system operates in pure offset-major mode. As density increases, larger bundles become profitable. The transition is governed by the density-aware configuration described next.

3.3 Density-Aware Configuration

The execution model exposes a low-dimensional configuration space: tile size, bundle size, and ordering strategy. The optimal effective throughput at a given active ratio corresponds to a specific point in this space, and the best point shifts as density changes.

ActiMM exploits the inference scenario (fixed model + fixed hardware) to resolve this choice entirely offline via a two-level profiling procedure. *Level 1*: for each layer in the model, sweep the configuration space across a range of active ratios and record the optimal configuration at each ratio, producing a per-layer *config lookup table*. *Level 2*: establish a mapping from the input point cloud to the per-layer active ratio (e.g., via voxel count). At inference time, the host reads the input's active ratio, indexes into the per-layer lookup tables, and selects the configuration for every layer—a host-only operation with zero GPU overhead and no device-to-host synchronization. No runtime schedule construction is required.

References

[1] Yukang Chen, Jianhui Liu, Xiangyu Zhang, Xiaojuan Qi, and Jiaya Jia. 2023. VoxelNeXt: Fully Sparse VoxelNet for 3D Object Detection and Tracking. In *CVPR*.

[2] Christopher Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *CVPR*.

[3] Lue Fan, Feng Wang, Ziqi Pang, Lei Zhu, Zhicheng Su, Fang Chen, Naiyan Wang, and Zhaoxiang Zhang. 2022. Embracing Single Stride 3D Object Detector with Sparse Transformer. In *CVPR*.

[4] Yujun Lin, Zhekai Wang, Yingyan Ding, and Song Han. 2021. PointAcc: Efficient Point Cloud Accelerator. In *MICRO*.

[5] Haotian Tang, Zhijian Liu, Shengyu Zhao, Yujun Lin, Ji Lin, Hanrui Wang, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *MLSys*.

[6] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. PCEngine: Efficient Point Cloud Inference Engine. In *MLSys*.

[7] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *MICRO*.

[8] Yan Yan, Yuxing Mao, and Bo Li. 2018. SECOND: Sparsely Embedded Convolutional Detection. *Sensors*.